Torneo de Komikan Concurrente

Estudiante: Matías Cáceres (matias.caceres6@mail.udp.cl) • **Carrera:** Ing. Civil en Informática y Telecomunicaciones
Asignatura: Sistemas Operativos • **Profesor:** Jonathan Frez • Noviembre 2026

1. Esquema de Sincronización y Garantía de Exclusión Mutua

El sistema modela una simulación en tiempo real estructurada en dos niveles desacoplados de concurrencia: la administración global del torneo (nivel macro) y la competencia física en cada partida independiente (nivel micro), implementados mediante el estándar POSIX Threads (`pthreads`).

1.1. Control de Admisión y Asignación de Mesas (Semáforos POSIX y Mutex Global)

Para arbitrar el acceso de los N clientes a un máximo estricto de M partidas concurrentes, el servidor (*Matchmaker* en `torneo.cpp`) emplea un semáforo contador POSIX (`sem_t sem_mesas`) inicializado con el valor M. Los jugadores se emparejan secuencialmente en duplas (Puma y Perros). Antes de instanciar e iniciar una nueva mesa, el hilo despachador ejecuta `sem_wait(&sem_mesas)`.

Si las M mesas se encuentran ocupadas, el hilo entra de inmediato en un estado de espera pasiva administrado por el planificador del kernel, descartando por completo el sondeo activo (*busy waiting*) y el desperdicio de ciclos de CPU. Ante interrupciones por señales (`EINTR`), la llamada gestiona el reintento de forma segura. Al culminar una partida por victoria o abandono, el hilo gestor ejecuta `sem_post(&sem_mesas)`, despertando deterministamente a la siguiente dupla en cola. Asimismo, el registro, consulta y eliminación de partidas en curso se protege atómicamente mediante el cerrojo `mutex_mesas_activas`, impidiendo colisiones en la estructura global del servidor.

1.2. Exclusión Mutua en el Tablero de Juego (Mutex por Mesa)

Cada partida se disputa sobre un grafo de 28 casillas (grilla de 25 nodos ortogonales más un trono superior de 3 nodos). En cada mesa coexisten 13 hilos concurrentes que deciden y actúan en tiempo real sin turnos fijos: 1 hilo para el Puma y 12 hilos independientes para los Perros. Para evitar condiciones de carrera (*race conditions*) al consultar la validez de casillas contiguas o disputar la ocupación simultánea de un mismo nodo libre, cada mesa administra su propio cerrojo `pthread_mutex_t mutex_tablero`.

Toda pieza que resuelve efectuar un movimiento adquiere el cerrojo de su mesa, verifica atómicamente la adyacencia y disponibilidad del nodo destino sobre el estado vigente en memoria, aplica el cambio de posición (o el retiro de la víctima si ocurre captura) y libera el cerrojo de inmediato. Este mecanismo preserva la invariante topológica: **en ningún instante pueden coexistir dos piezas en la misma casilla**, ni consumirse un perro previamente retirado.

1.3. Sincronización de Salida y Manejo Asíncrono de Señales (*Async-Signal Safety*)

Para impedir el entrelazado corrupto de texto y representaciones ASCII del tablero cuando múltiples mesas paralelas escriben a la terminal, se dispuso el cerrojo global `mutex_consola`. Por su parte, la interrupción por `SIGINT` (`Ctrl+C`) se diseñó bajo estándares estrictos de seguridad asíncrona: la rutina capturada con `sigaction` no ejecuta llamadas bloqueantes ni funciones no reentrantes (`printf`, `malloc`); únicamente conmuta la variable atómica `volatile sig_atomic_t terminar_programa = 1` y emite un mensaje de bajo nivel mediante `write()`. La propagación de la detención a las mesas activas y la sincronización exhaustiva mediante `pthread_join()` se ejecutan exclusivamente en el hilo principal, garantizando el cierre ordenado sin hilos zombis ni fugas de memoria.

2. Análisis y Mitigación de Riesgos de Concurrencia

La naturaleza continua y asíncrona de las decisiones exige blindar la arquitectura ante las anomalías clásicas de los sistemas multihilo:

2.1. Prevención de Interbloqueos (Deadlock y Condiciones de Coffman)

Un interbloqueo requiere la concurrencia simultánea de las cuatro condiciones de Coffman. En este diseño se destruyó de raíz la condición de **Espera Circular** mediante dos principios arquitectónicos:

- **Aislamiento total entre mesas:** Cada hilo de pieza accede únicamente al `mutex_tablero` de su propia instancia. Dado que las mesas son estructuras disjuntas en memoria y no existe comunicación cruzada, el Grafo de Asignación de Recursos (RAG) entre partidas es completamente acíclico y disconexo.
- **Jerarquía estricta de adquisición de cerrojos (*Lock Ordering*):** Cuando una pieza o el árbitro genera un evento de salida en consola, adquiere `mutex_consola` estando retenido `mutex_tablero`. Para asegurar la ausencia absoluta de ciclos de espera, se impone la regla jerárquica estricta: `mutex_tablero` $\prec$ `mutex_consola`. Ningún hilo solicita jamás `mutex_tablero` mientras retiene `mutex_consola`, garantizando que las aristas de asignación y solicitud sigan un orden topológico unidireccional libre de dependencias circulares.

2.2. Evasión de Livelock (Bloqueo Activo)

El livelock se manifestaría si dos o más piezas compitieran reactivamente en ciclos simétricos indefinidos sin lograr progreso sustancial (por ejemplo, oscilación perpetua e idéntica de piezas contiguas). Para quebrar cualquier sincronía espuria o resonancia armónica, cada pieza aplica un retardo estocástico desacoplado, muestreado uniformemente en el intervalo [`retardo_min_ms`, `retardo_max_ms`] antes de cada jugada. Esta desincronización pseudoaleatoria rompe el

determinismo de contención, induciendo convergencia inexorable hacia un desenlace definitivo (6 capturas o inmovilización del Puma).

2.3. Mitigación de Inanición (Starvation)

La inanición ocurriría si un hilo particular quedara relegado indefinidamente del tablero debido a la constante monopolización de los otros 12 hilos competidores. Para asegurar progreso equitativo:

- **Cesión forzada del procesador:** Tras cada intento de movimiento (sea exitoso o infructuoso), los hilos duermen invocando `usleep`. Esto impide la monopolización de la CPU por parte de hilos con alta frecuencia de ráfaga y provee al planificador del sistema operativo la ventana indispensable para despachar equitativamente al resto de los hilos encolados en el cerrojo.
- **Descentralización espacial:** Al estar los 12 perros distribuidos en distintas zonas de la grilla de 28 posiciones, la contención por adyacencias específicas se dispersa geográficamente, reduciendo rechazos reiterados sobre un mismo sector.

3. Evaluación de Complejidad, Throughput y Granularidad

La selección de la granularidad en los primitivos define el balance óptimo entre rendimiento, costo de sincronización y riesgo de fallo:

3.1. Granularidad por Mesa vs. Cerrojo Global del Torneo

Una solución simplista habría utilizado un único mutex global para todo el proceso. Dicho esquema serializaría la ejecución de las M mesas, colapsando el torneo a un modelo secuencial y anulando por completo las capacidades de paralelismo real en arquitecturas multi-core. La adopción de un cerrojo independiente por mesa aísla la contención, permitiendo que el *throughput* global escale de manera cuasilineal respecto a los núcleos de cómputo disponibles.

3.2. Granularidad por Mesa vs. Bloqueo por Casilla Individual

Se evaluó la alternativa de asignar un cerrojo exclusivo a cada una de las 28 casillas del grafo. Si bien esta granularidad fina eleva teóricamente la concurrencia interna, en la práctica introduce penalizaciones prohibitivas:

- **Sobrecarga de sincronización (*Locking Overhead*):** En un espacio reducido de 28 nodos, el costo de las llamadas al sistema para bloquear y liberar múltiples posiciones supera ampliamente el tiempo de cómputo requerido para validar el movimiento en memoria compartida.
- **Riesgo crítico de Deadlock en capturas:** La maniobra de salto del Puma involucra la adquisición simultánea y atómica de tres nodos contiguos (origen, víctima intermedia y destino libre). Coordinar tres cerrojos en tiempo real exigiría protocolos complejos de ordenamiento de recursos (*lock ordering*), disparando el riesgo de interbloqueo frente a perros que disputen esas mismas casillas.

Estrategia Evaluada	Concurrencia Efectiva	Sobrecarga de Sincronización	Riesgo de Interbloqueo
Cerrojo Global Único	Nula (Serialización forzada)	Mínima (1 cerrojo global)	Nulo
Bloqueo por Casilla	Alta intra-mesa	Muy Alta (múltiples llamadas)	Alto (capturas de 3 nodos)
Mutex por Mesa (Adoptada)	**Óptima (Paralelismo real M)**	**Despreciable (1 cerrojo/jugada)**	**Nulo (Grafo RAG acíclico)**

3.3. Conclusiones y Balance de Arquitectura

La integración de un semáforo contador POSIX para el control de admisión en el servidor, complementado por un cerrojo `mutex_tablero` por mesa y un protocolo jerárquico estricto hacia `mutex_consola`, proporciona una solución con máxima solidez formal. Garantiza la integridad de la memoria compartida, previene anomalías de concurrencia y maximiza la tasa de procesamiento en sistemas multi-hilo.